

Document Number	P2560R0
Date	2022-02-23
Reply-to	Matúš Chochlík chochlik@gmail.com
Audience	SG7

Reflection – Metaobjects

value-based

Reflection uses constant values of a single built-in type:

```
info x = reflect(argument);
```

Reflection uses constant values of multiple types:

```
auto x = reflect(argument);
```

type-based

The actual (implementation-defined) type might be:

```
template <info X>  
struct __metaobject {  
    constexpr operator info() const {  
        return X;  
    }  
};
```

Reflection – APIs

value-based

Either purely consteval or template functions with non-type template parameters:

```
constexpr auto foo(info mo);
```

```
template <info MO>  
constexpr auto bar();
```

type-based

constexpr template functions taking metaobjects as function arguments:

```
constexpr auto foo(metaobject auto mo);  
constexpr auto bar(metaobject auto mo);
```

```
template <typename T>  
concept metaobject = unspecified;
```

Reflection – Implementation

value-based

Very fast to compile:

```
constexpr auto foo(info mo);
```

Somewhat slower to compile:

```
template <info MO>  
constexpr auto bar();
```

type-based

Very fast to compile, but requires `constexpr` conversion from metaobject to `info`:

```
constexpr auto foo(info mo);
```

Slower to compile:

```
template <info MO>  
constexpr auto bar(__metaobject<MO> mo);
```

Reflection – Usage

value-based

Dual syntax:

```
use(foo(reflect(argument)));
```

or

```
use(bar<reflect(argument)>());
```

When to use which?

type-based

Uniform syntax:

```
use(foo(reflect(argument)));
```

and

```
use(bar(reflect(argument)));
```

Reflection – Containers

value-based

Vectors, etc. must be fixed to work in consteval

```
vector<info> x = members_of(...);  
vector<info> y = bases_of(...);
```

Can be used with some STL algorithms, unless splicing is involved

type-based

Containers (sequences) are metaobjects themselves

```
auto x = get_data_members(...);  
auto y = get_base_classes(...);
```

Have their own implementation of reflection-related algorithms, splicing is no problem

Reflection – Pros

value-based

- Faster to compile
- Uses less resources to compile

type-based

- Consistent and unified API
- More friendly to generic programming
- Plays better with ADL
- Better usability
- Easier to teach

Reflection – Cons

value-based

- Inconsistent API
- The `foo(...)` vs. `bar<...>()` syntax makes it less generic
- Rules when to use which, are sort of complicated and may look arbitrary
- More complicated to teach
- Issues with ADL on NTTPs

type-based

- Slower to compile
- Uses more resources to compile

Usability issues – the dual value-based API

```
// span<meta::info>, vector<meta::info>  
auto mem = members_of(^T);  
auto func = // some callable, example later
```

The following is possible only for a subset of possible reflection operations:

```
std::count_if1(mem.begin(), mem.end(), func);
```

Specifically the func cannot use splicing, because count_if will call it as:

```
func(element);
```

and not as:

```
func<element>();
```

¹or any other of countless possible algorithms

Usability issues – writing generic algorithms

Users² will want to write their own reusable algorithms, that take other functions³ as their arguments:

```
constexpr void my_reusable_algo(  
    span<meta::info> s,  
    function<bool(meta::info)> predicate,  
    function<void(meta::info)> function) {  
    for(auto e : s) {  
        if(predicate(e) && something_else(e)) {  
            function(e);  
        }  
    }  
}
```

predicate, something_else and function *cannot* do splicing...

²and library authors

³predicates, transforms, etc.

Usability issues – supporting splicing

...to support splicing we'd have to:

```
template <auto S>
constexpr void my_reusable_algo(
    auto predicate,
    auto function) {
    template for(auto e : S) {
        if(predicate<e>() && something_else<e>()) {
            function<e>();
        }
    }
}
```

making everything a template. But then this becomes slower to compile, partially defeating one of the main points of this API.

BTW, why so much focus on splicing? – some anecdotes...

- Out of these use-cases⁴⁵
 - enum / string conversion,
 - serialization and deserialization,
 - parsing of command line arguments into a config structure,
 - RPC stubs and skeletons,
 - generic wrapper for a REST API,
 - automated registering with a scripting engine,
 - generating UML diagrams from code,
 - fetching and converting data from an SQL database,
 - generating SQL queries from the names in an “interface” class,
 - implementation of the factory pattern.
- All but one⁶ required splicing
- Various forms of splicing are *very* common in use-cases

⁴all implemented here: <https://github.com/matus-chochlik/mirror>

⁵and there is a whole other presentation about the details

⁶UML generation

What are we trying to do?

Determine what is the actual overhead of this:

```
template <info X>
struct __metaobject {
    consteval operator info() const { return X; }
};
concept metaobject = unspecified;

consteval auto foo(info mo);
consteval auto bar(metaobject auto mo);
```

compared to this:

```
consteval auto foo(info mo);

template <info MO>
consteval auto bar();
```

in a real-life scenario.

How to materialize 100'000s of metaobjects?

Use a shell script...

```
L=100 # number of repeats
S=1000 # sampling step size
for l in $(seq 1 ${L})
do
  N=$((l * S))
  # factorize N into three integers
  D=...; E=...; F=...
```

...to generate a C++ source file...

```
int main() {
  return bool(qux(make_index_sequence<${D}>{})) ? 0 : 1;
}
```

..., compile and measure:

```
time $(CXX) $(CXXFLAGS) -o /dev/null $<
done
```

The boilerplate – level 1

```
template <size_t ... K>
constexpr auto qux(index_sequence<K...>) {
    return ( ... + baz(
        integral_constant<size_t, K>{},
        make_index_sequence<${E}>{}));
}
```


The boilerplate – level 2

```
template <size_t K, size_t ... J>
constexpr auto baz(
    integral_constant<size_t, K>,
    index_sequence<J...>) {
    return ( ... + bar(
        integral_constant<size_t, K>{},
        integral_constant<size_t, J>{},
        make_index_sequence<${F}>{}));
}
```

The boilerplate – level 3

```
template <size_t K, size_t J, size_t ... I>
constexpr auto bar(
    integral_constant<size_t, K>,
    integral_constant<size_t, J>,
    index_sequence<I...>) {
    // Simulate the metaobject "id" as:
    // MOID =
    //     K * $((N / D)) +
    //     J * $((N / (D * E))) +
    //     I;
    return /* Do something with MOID... */
}
```

The baseline

Just sum the *MOID* values at compile-time

```
template <size_t K, size_t J, size_t ... I>
constexpr auto bar(
    integral_constant<size_t, K>,
    integral_constant<size_t, J>,
    index_sequence<I...>) {
    return (... +
        K * $((N / D)) +
        J * $((N / (D * E))) +
        I);
}
```

Measure how long does this take to compile and subtract from “real” measurements.

Type-based metaobject & template function

```
template <size_t M>
struct wrapper {
    consteval operator size_t() const {
        return M;
    }
};
```

```
template <size_t M>
consteval size_t foo(wrapper<M> w) {
    return w;
}
```

```
return ( ... + foo(wrapper<MOID>{}));
```

Type-based metaobject & consteval function

```
template <size_t M>
struct wrapper {
    consteval operator size_t() const {
        return M;
    }
};

consteval size_t foo(size_t m) {
    return m;
}
```

```
return ( ... + foo(wrapper<MOID>{ }));
```

Value-based metaobject & template function

```
template <size_t M>
constexpr size_t foo() {
    return M;
}
```

```
return ( ... + foo<MOID>());
```

Value-based metaobject & consteval function

```
constexpr size_t foo(size_t m) {
    return m;
}
```

```
return ( ... + foo(MOID));
```

Test hardware

- Old desktop⁷:
 - i5-2400U @ 3.10GHz (4 cores)
 - 24GB RAM
- Corporate dev laptop⁸:
 - i7-1185G7 @ 3.0GHz (8 cores)
 - 32GB RAM
- Mid-range gaming laptop⁹:
 - AMD Ryzen7 4800HS (16 cores)
 - 16GB RAM
- RPi 4B¹⁰
 - ARM v7l
 - 4GB RAM

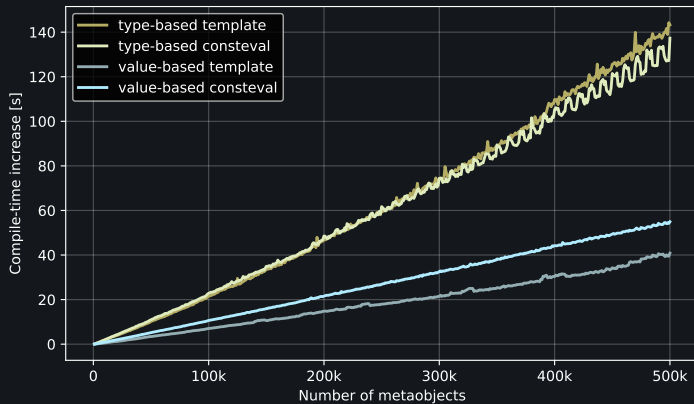
⁷2010

⁸2021

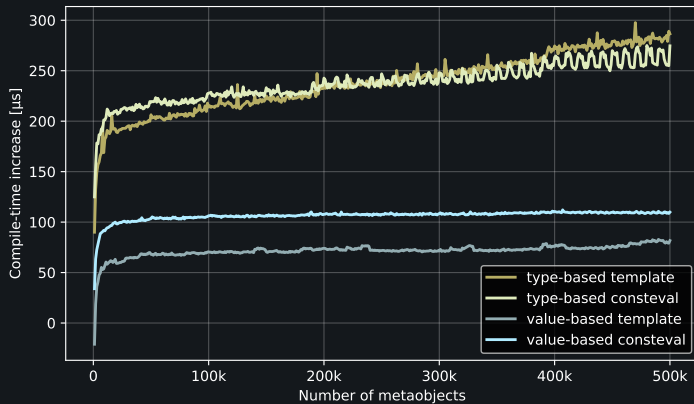
⁹2019

¹⁰timeless

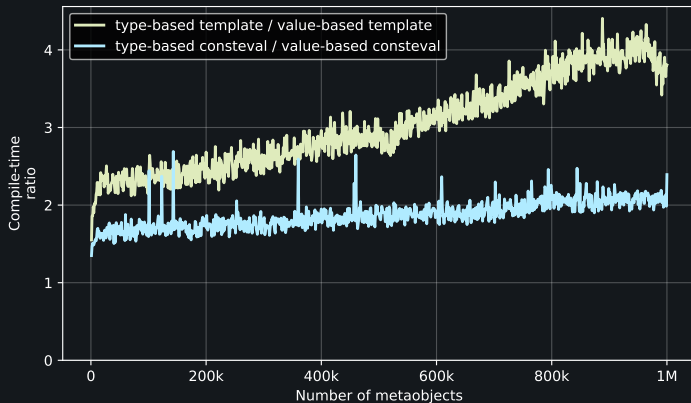
i5-2400 – compile time increase per N metaobjects



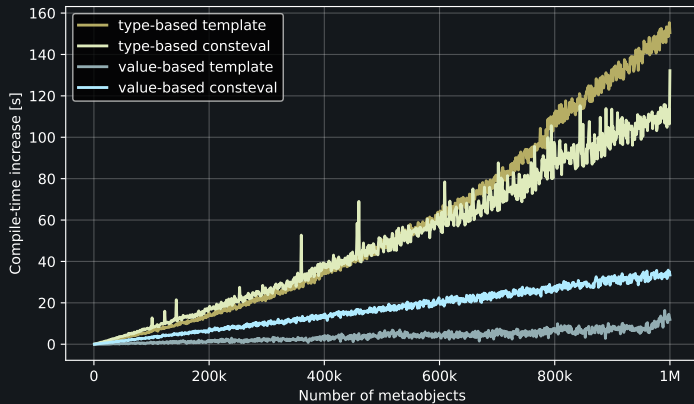
i5-2400 – compile time increase per 1 metaobject



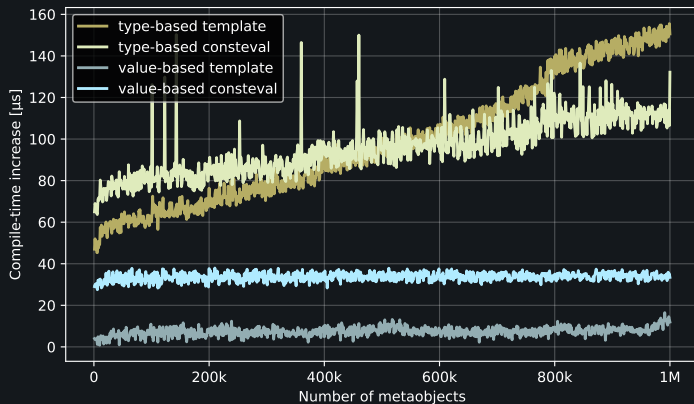
i5-2400 – How much faster is value-based vs. type-based



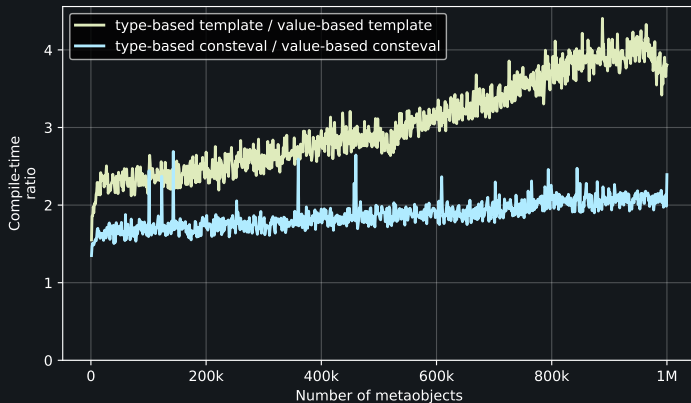
i7-1185 – compile time increase per N metaobjects



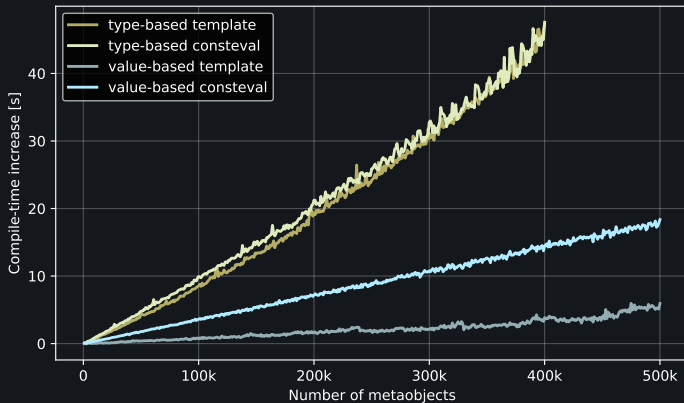
i7-1185 – compile time increase per 1 metaobject



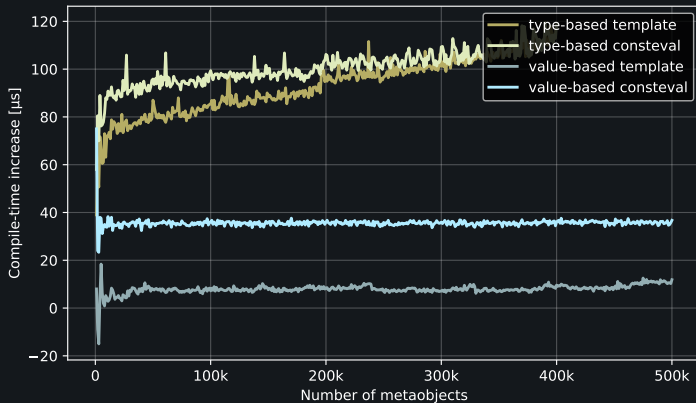
i7-1185 – How much faster is value-based vs. type-based



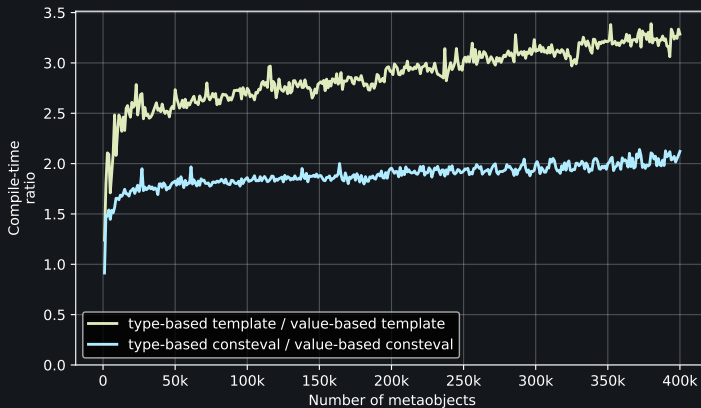
Ryzen7-4800HS – compile time increase per N metaobjects



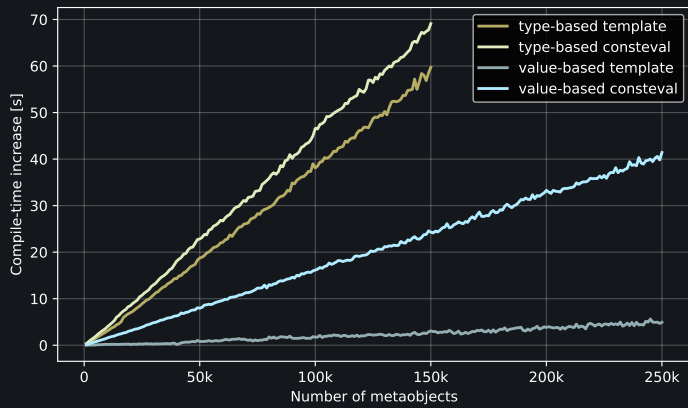
Ryzen7-4800HS – compile time increase per 1 metaobject



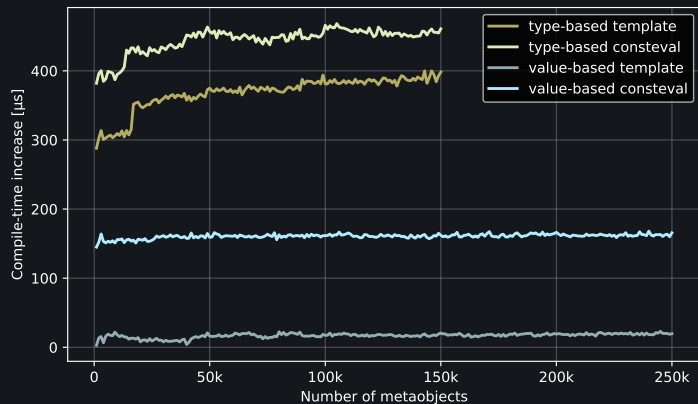
Ryzen7-4800HS – How much faster is value-based vs. type-based



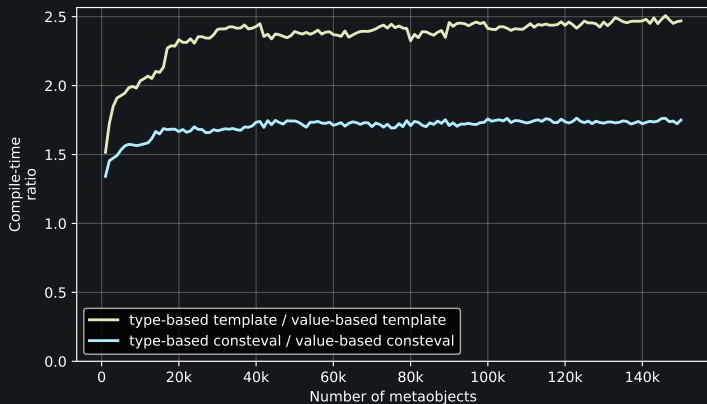
ARMv7 – compile time increase per N metaobjects



ARMv7 – compile time increase per 1 metaobject



ARMv7 – How much faster is value-based vs. type-based



What about executable sizes?

- This is boring. . .
- When the reflection-related functions are `constexpr`, the executable size stays the same regardless of the representation of metaobjects or their count
- The test source code shown above always compiles into an executable roughly 16kB in size

But, what if – we tried something different. . .

- Instead of that template gizmo instantiating metaobjects in one place in the source
- Just generate a lot of source code with many separate metaobject operations
- As in. . .

Type-based metaobject & template function

```
template <size_t M>
struct wrapper {
    consteval operator size_t() { return M; }
};
template <size_t M>
consteval size_t foo(wrapper<M> w) {
    return w;
}
```

```
consteval int bar() {
    return static_cast<int>(
        foo(wrapper<1Z>{}) +
        foo(wrapper<2Z>{}) +
        // ...
        foo(wrapper<NZ>{}));
}
```

Type-based metaobject & consteval function

```
template <size_t M>
struct wrapper {
    consteval operator size_t() { return M; }
};

consteval size_t foo(size_t m) {
    return m;
}
```

```
consteval int bar() {
    return static_cast<int>(
        foo(wrapper<1Z>{}) +
        foo(wrapper<2Z>{}) +
        // ...
        foo(wrapper<NZ>{}));
}
```


Value-based metaobject & template function

```
template <size_t M>
constexpr size_t foo() {
    return M;
}
```

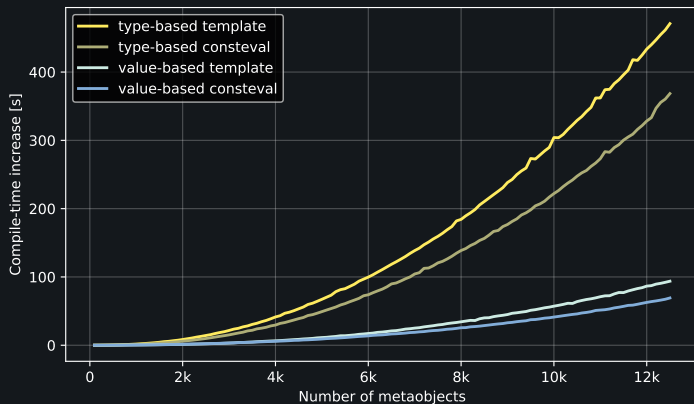
```
constexpr int bar() {
    return static_cast<int>(
        foo<1Z>()+
        foo<2Z>()+
        // ...
        foo<NZ>());
}
```

Value-based metaobject & consteval function

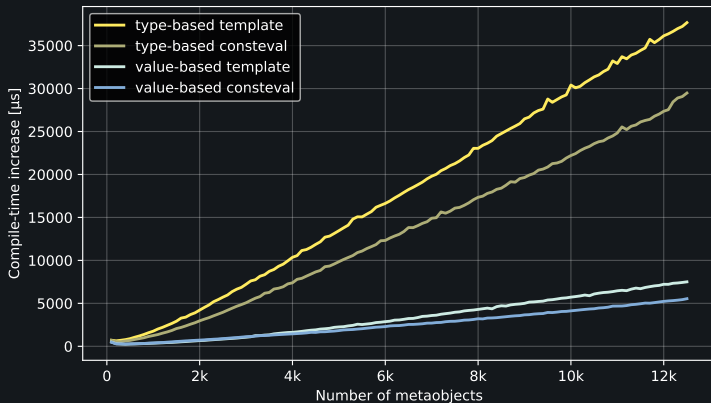
```
constexpr size_t foo(size_t m) {
    return m;
}
```

```
constexpr int bar() {
    return static_cast<int>(
        foo(1Z)+
        foo(2Z)+
        // ...
        foo(NZ));
}
```

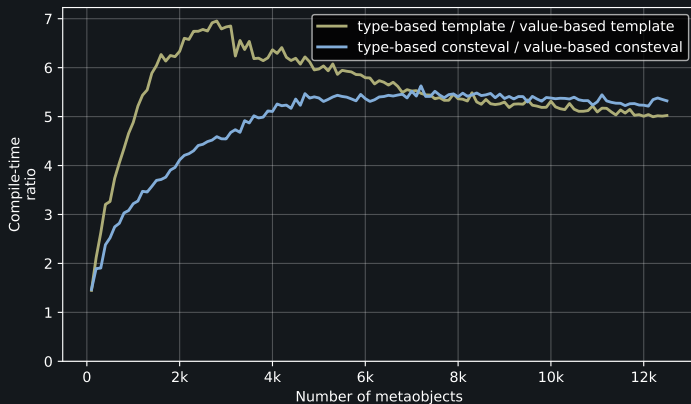
i5-2400 – compile time increase per N metaobjects



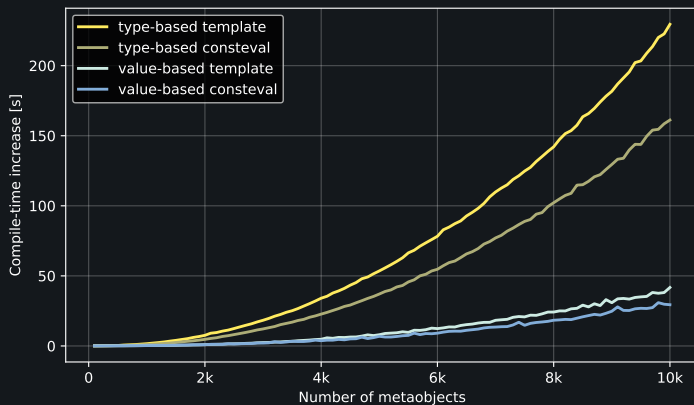
i5-2400 – compile time increase per 1 metaobject



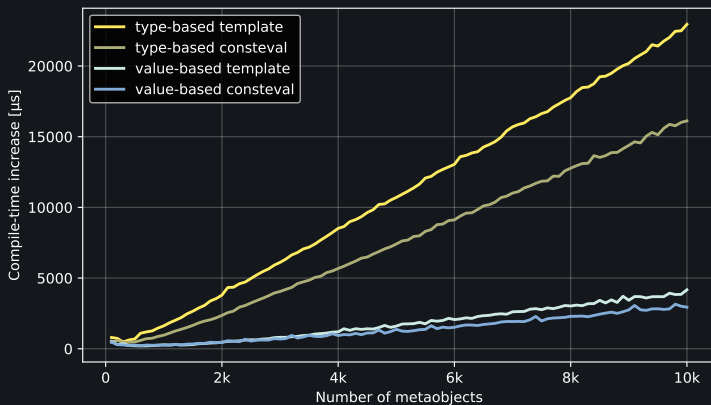
i5-2400 – How much faster is value-based vs. type-based



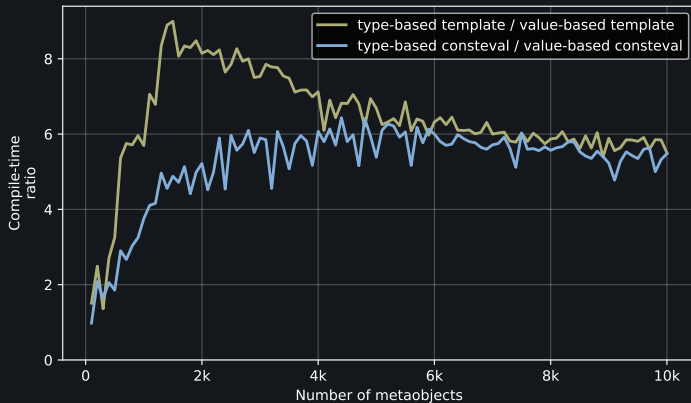
Ryzen7-4800HS – compile time increase per N metaobjects



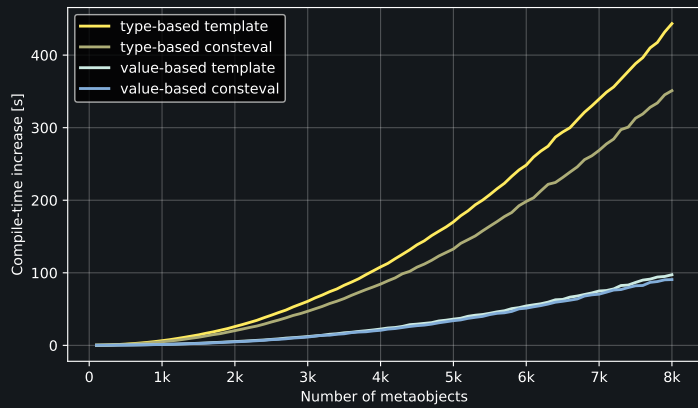
Ryzen7-4800HS – compile time increase per 1 metaobject



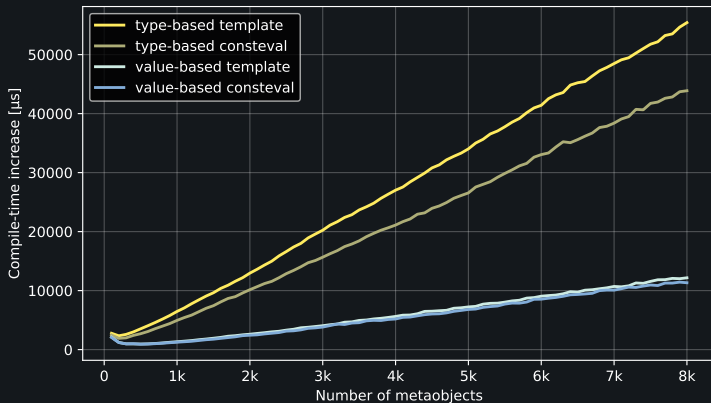
Ryzen7-4800HS – How much faster is value-based vs. type-based



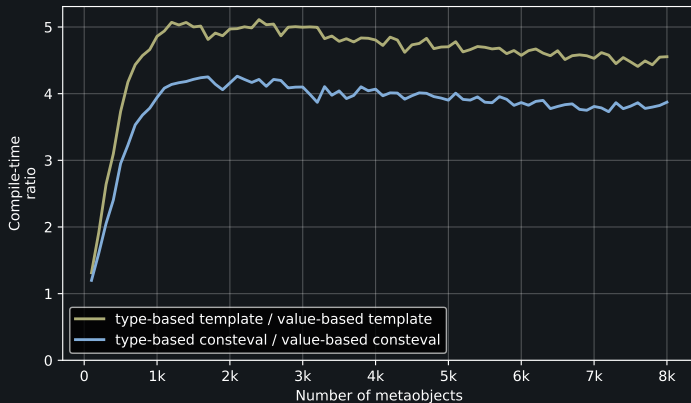
ARMv7 – compile time increase per N metaobjects



ARMv7 – compile time increase per 1 metaobject



ARMv7 – How much faster is value-based vs. type-based



- Just some guesses. . .
 - In the second setup, each metaobject is an individually-parsed expression
 - Different than just multiple instantiations
 - Different source locations, etc.
- Even in the value-based cases the compile-times grow non-linearly
- This is *not* how you typically use reflection in real-life scenarios

Representing the reflection “operator”

- Above we have just used integer literals to represent metaobject “ids”
- What if we used a template function¹¹ to simulate the reflection expression
- What would be the effects on the compile-times?

¹¹with NTTP

Representing the reflection “operator” – (cont.)

value-based

```
template <size_t I>
constexpr auto reflect() {
    return I;
}
```

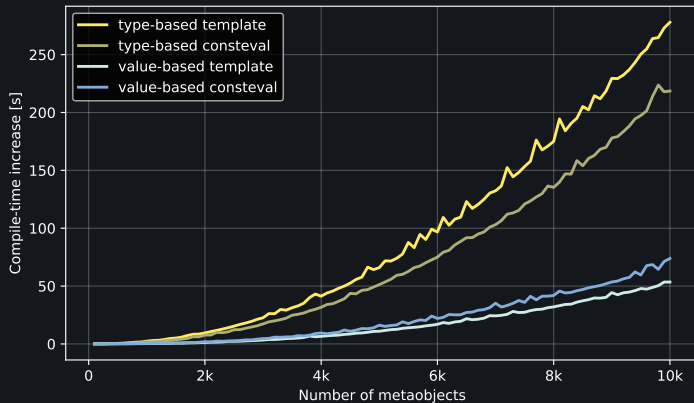
```
foo(reflect<NZ>());
// and
foo<reflect<NZ>()>();
```

type-based

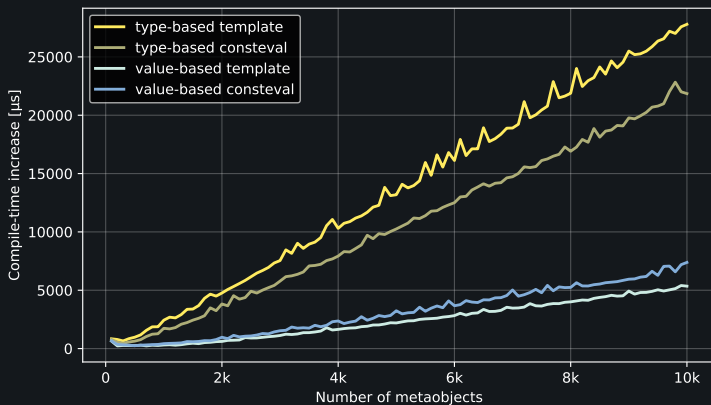
```
template <size_t I>
constexpr wrapper<I> reflect() {
    return {};
}
```

```
foo(reflect<NZ>()));
```

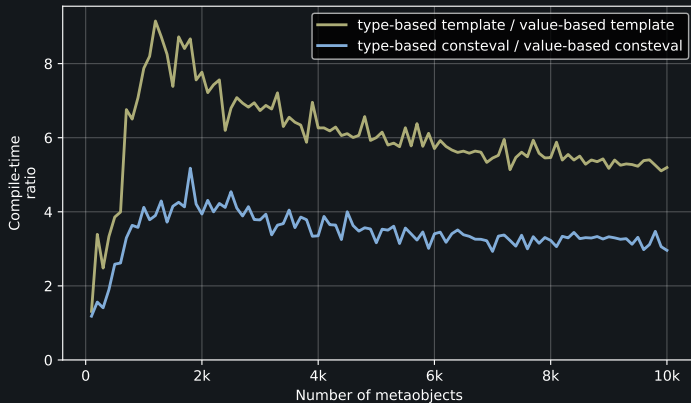
Ryzen7-4800HS – compile time increase per N metaobjects



Ryzen7-4800HS – compile time increase per 1 metaobject



Ryzen7-4800HS – How much faster is value-based vs. type-based



Estimating number of declarations in clang

Let's try *documented* declarations

Edit doxygen-cfg.in:

```
- GENERATE_XML    = NO
+ GENERATE_XML    = YES
```

Configure:

```
cmake \
  -DLLVM_ENABLE_DOXYGEN=On \
  ...
```

Generate Doxygen docs:

```
ninja doxygen-clang
```

Merge into a single XML file clang.xml:

```
xsltproc combine.xslt index.xml > clang.xml
```


- That's for version 15.0.0
- Around FEB-05-2022
- Round that up to 400'000, 500'000 or even 1'000'000
- Let's assume we want to reflect every single declaration

Clean build of clang

Edit toolchain.cmake:

```
set(LLVM_USE_LINKER lld)  
set(CMAKE_EXE_LINKER_FLAGS -fuse-ld=${LLVM_USE_LINKER})  
set(CMAKE_SHARED_LINKER_FLAGS -fuse-ld=${LLVM_USE_LINKER})
```

Configure:

```
cmake \  
  -DLLVM_ENABLE_PROJECTS="clang;clang-tools-extra" \  
  -DLLVM_ENABLE_RUNTIMES="libcxx;libcxxabi" \  
  -DLLVM_TOOLCHAIN_FILE="toolchain.cmake" \  
  ...
```

Build and measure elapsed time:

```
time ninja install install-cxx install-cxxabi
```

Clean build of clang

Results:

CPU:	i5-2400	i7-1185	Ryzen 7
real	122m25,943s	66m59,909s	34m45,899s
user	433m50,123s	510m55,382s	525m16,660s
sys	11m22,881s	12m52,287s	17m5,738s

Added, rounded and converted to seconds:

CPU:	i5-2400	i7-1185	Ryzen 7
real-time	7346s	4020s	2086s
cpu-time (user+sys)	27313s	31427s	32543s

Conclusions

- The typical compile-time overhead of materializing a metaobject is on the order of tens or hundreds of microseconds
- The type-based metaobject representation is between 2x and 6x¹⁷ slower to compile compared to the purely value-based representation

¹⁷in the worst “copy-paste” use-case

The big question

Is the improvement in compile-time
worth the decrease in usability
of the value-based reflection API?